in a `data` attribute of the element. We could access these values manually using JavaScript like so:

```
document.getElementsByClassName("cities")[0].__data__
```

Returns a pointer to the object representing San Francisco

Later in this chapter we'll work with those values in a more sophisticated way using D3.

.ENTER() AND .EXIT()

When binding data to selections, there will be either more, less, or the same number of DOM elements as there are data values. When you have more data values than DOM elements in the selection, you trigger the `.enter()` function, which allows you to define behavior to perform for every value that doesn't have a corresponding DOM element in the selection. In our case, `.enter()` fires four times, because no DOM elements correspond to `"div.cities"` and our `incomingData` array contains eight values. When there are fewer data elements, then `.exit()` behavior is triggered, and when there are equal data values and DOM elements in a selection, then neither `.exit()` nor `.enter()` is fired.

.APPEND() AND .INSERT()

You'll almost always want to add elements to the DOM when there are more data values than DOM elements. The `.append()` function allows you to add more elements and define which elements to add. In our example, we add `<div>` elements, but later in this chapter we'll add SVG shapes, and in other chapters we'll add tables and buttons and any other element type supported in HTML. The `.insert()` function is a sister function to `.append()`, but `.insert()` gives you control over where in the DOM you add the new element. You can also perform an append or insert directly on a selection, which adds one DOM element of the kind you specify for each DOM element in your selection.

.ATTR()

You're familiar with changing styles and attributes using D3 syntax. The only thing to note is that each of the functions you define here will be applied to each new element added to the page. In our example, each of our four new `<div>` elements will be created with `class="cities"`. Remember that even though our selection referenced `"div.cities"`, we still have to manually declare that we're creating `<div>` elements and also manually set their class to `"cities"`.

.HTML()

For traditional DOM elements, you set the content with a `.html()` function. In the next section, you'll see how to set content based on the data bound to the particular DOM element.

2.2.2 Accessing data with inline functions

If you ran the code in the previous example, you saw that each `<div>` element was set with different content derived from the data array that you bound to the selection. You did this using an inline anonymous function in your selection that automatically provides access to two variables that are critical to representing data graphically: the data value itself and the array position of the data. In most examples you'll see these